# CSI3007 - Advanced Python Programming

## LAB ASSESSMENT - 1

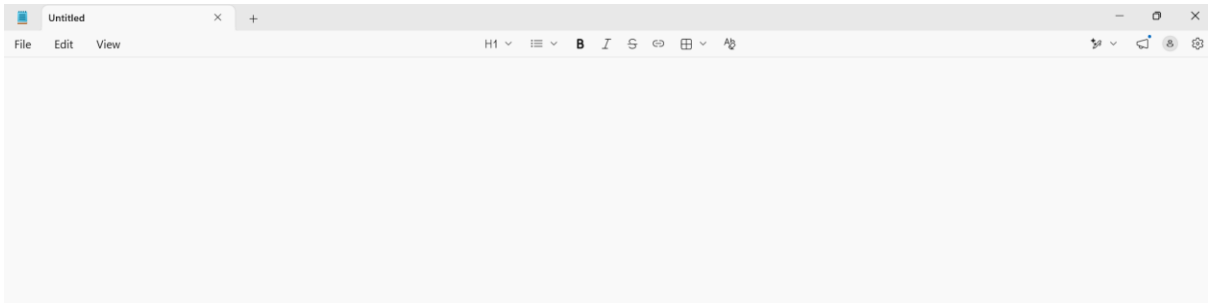
NAME : RAJAVELMURUGAN S

REG_NO : 23 MID0018

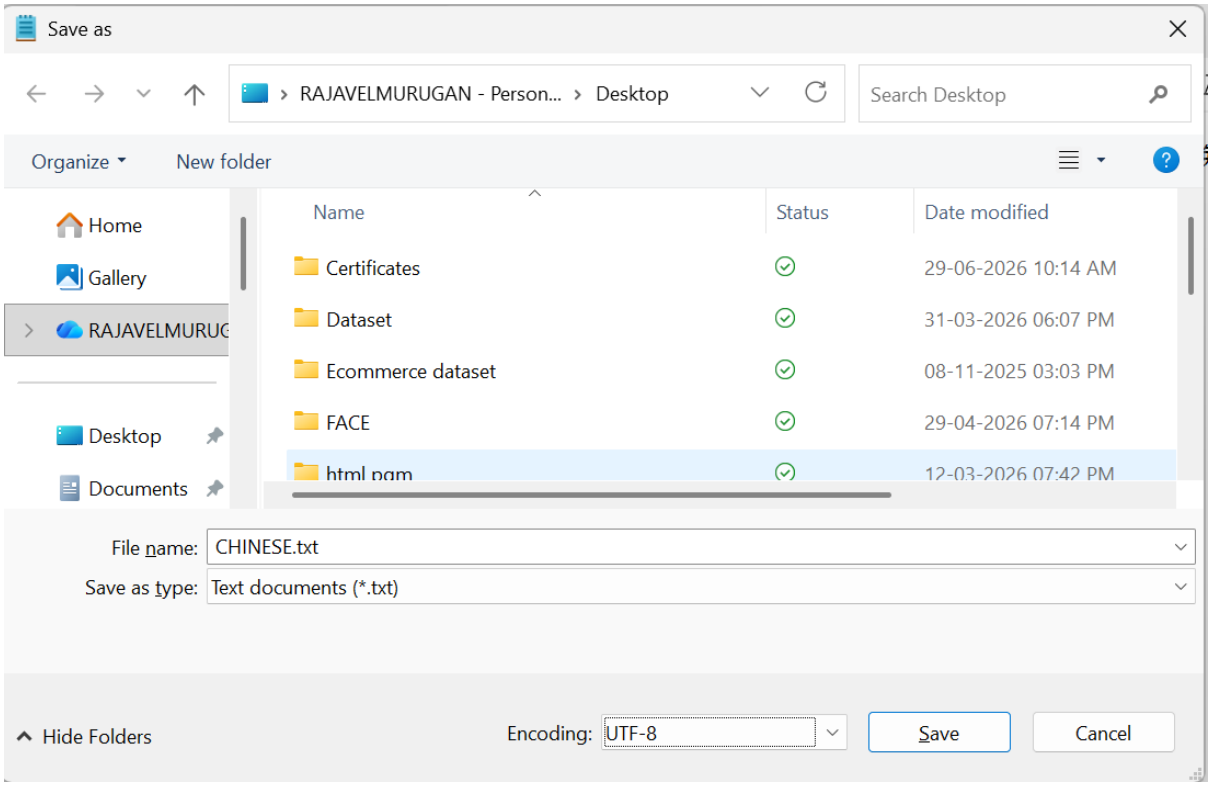
FACULTY : SHARMILA BANU K

LAB SLOT : L31+L32+L49+L50

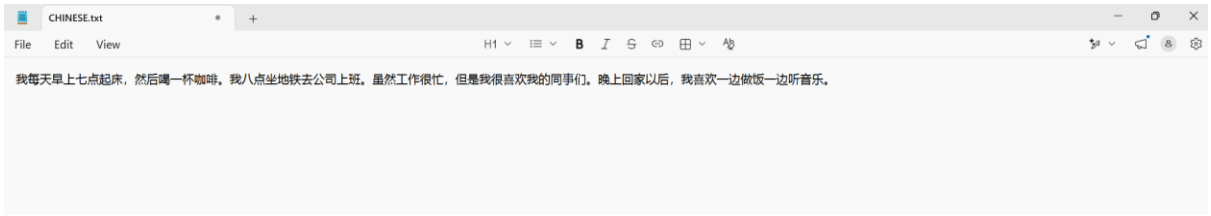
**Task 1: Chinese Words Character Encoding :**



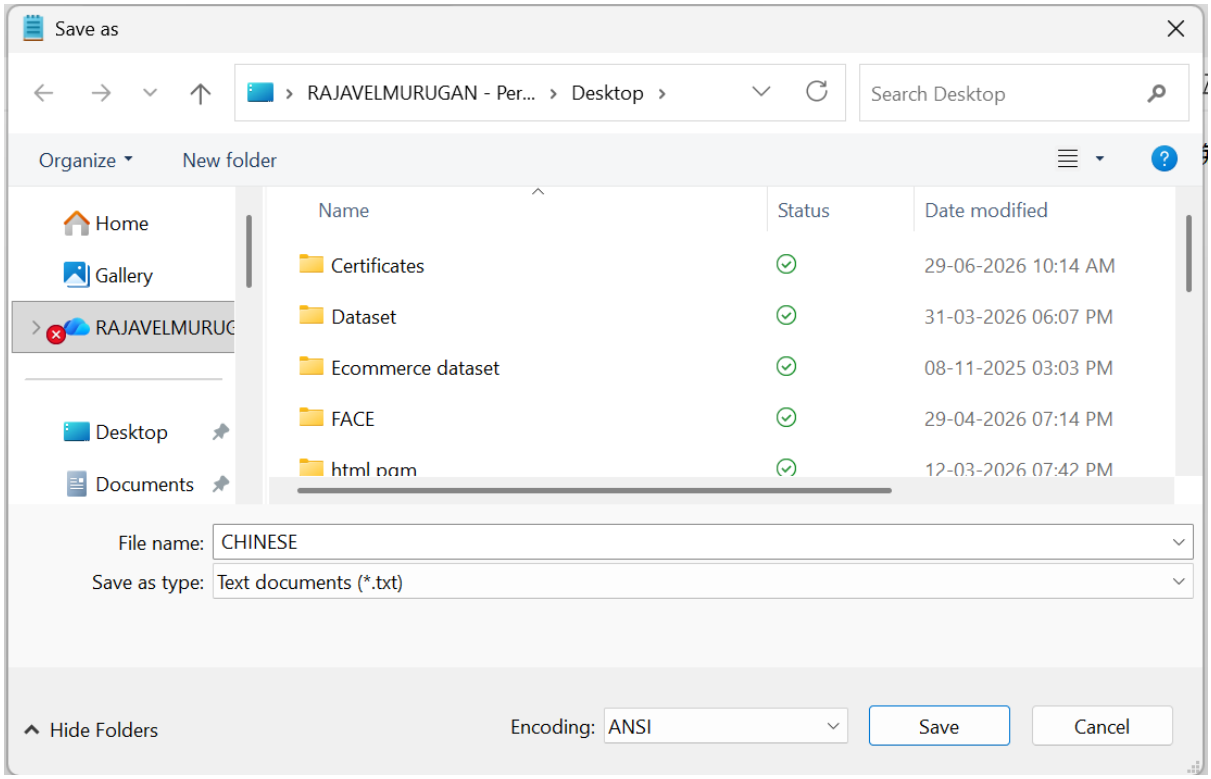

Open New text file



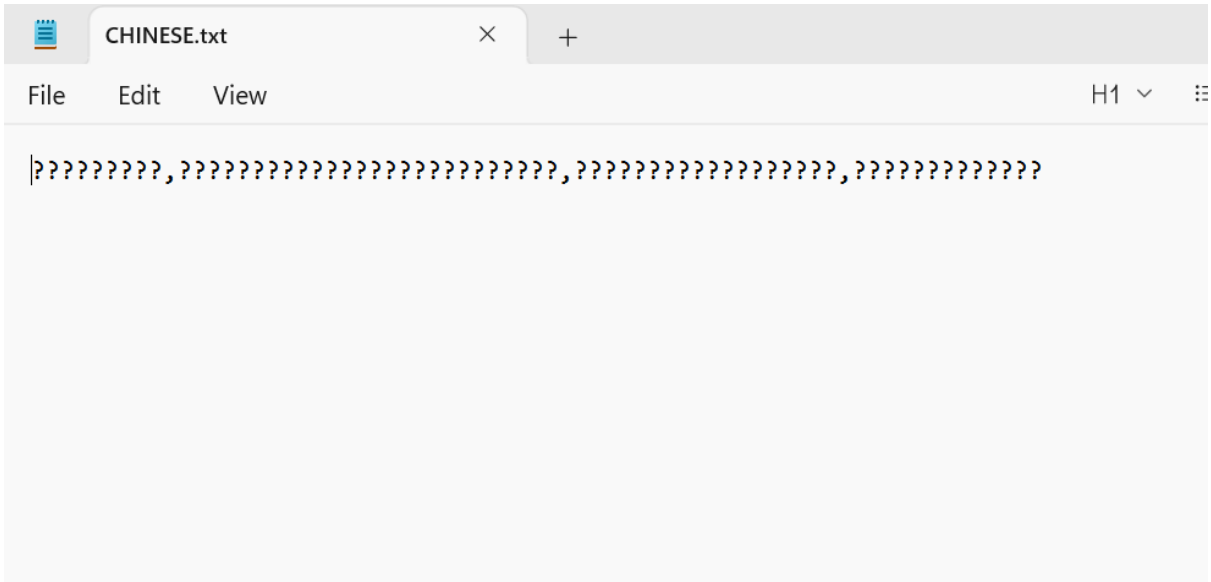

Save file as UTF-8 Encoding



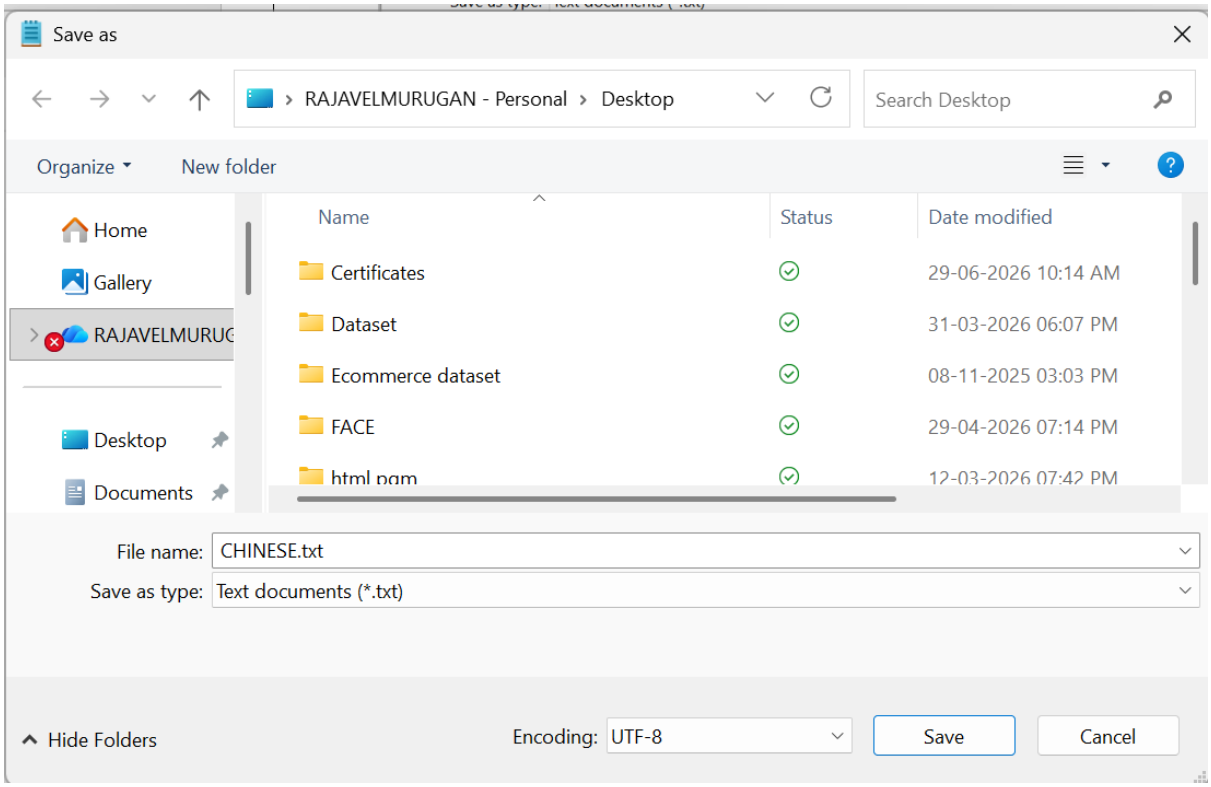

Copy paste the Chinese characters



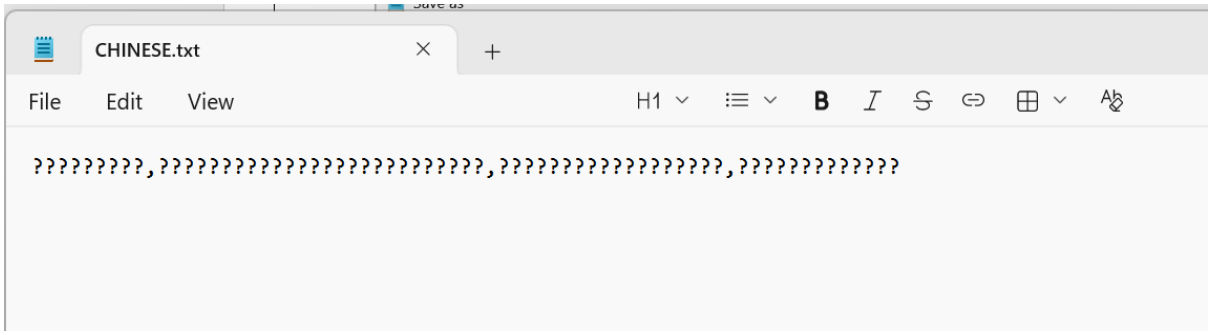

Now change the encoding to ANSI  Save the file




Once save the file in ANSI  encoding The characters will be “ ? “




Now change encoding to utf - 8




Characters will not revert from ‘ ? ‘ to Chinese characters

## Task 2: Working with PDF Files Using PyPDF

### Fetch pdf File

```
]: pip install pypdf
```

```
Collecting pypdf
  Downloading pypdf-6.14.2-py3-none-any.whl.metadata (7.2 kB)
Downloading pypdf-6.14.2-py3-none-any.whl (349 kB)
Installing collected packages: pypdf
Successfully installed pypdf-6.14.2
Note: you may need to restart the kernel to use updated packages.
```

```
]: from pypdf import PdfReader

reader = PdfReader(r"C:\Users\rajav\Downloads\VIT Downloads\Other Downloads\Assignments\Sample_pdf.pdf")  # Open the PDF file
```

### Extract text from pdf

```
: page = reader.pages[0]  # Read first page
text = page.extract_text()  # Extract text from page

print(text)  # Display the text
```

```
Explainable AI-Based Network Intrusion Detection Using
KNN and LightGBM on the UNSW-NB15 Dataset
Sairabanu J,
Professor,
SCOPE, VIT
VELLORE,
INDIA.
Santhoshkumar
M PG student
SCOPE, VIT,
VELLORE,
INDIA
Rajavel Murugan
S PG student
SCOPE, VIT,
VELLORE,
INDIA
Dharshini G PG
student    SCOPE,
VIT VELLORE,
INDIA
```

ABSTRACT
Cybersecurity threats have become a major concern today because of the growth of network-based applications. This paper presents an intrusion detection system (IDS) using machine learning techniques on the UNSW-NB15 dataset. The proposed system integrates feature engineering, feature selection and classification models such as K-Nearest neighbours (KNN) and LightGBM to detect malicious network activities with AI techniques like SHAP and LIME used to interpret model predictions , improving transparency and t rust. Experimental results demonstrate that the proposed system achieves high accuracy in classifying normal and traffic while providing meaningful insights into feature importance.
KEYWORDS
Intrusion Detection System, Machine Learning, UNSW-NB15, KNN, LightGBM, SHAP, LIME, Cybersecurity

## I. INTRODUCTION

In today's world which is running in internet is highly vulnerable to cyber threats making network security a critical concern. Intrusion Detection System (IDS) plays an important role in identifying and protecting network infrastructure. Traditional IDS m ethods which rely on signature-based detection, which is less accurate as compared to modern IDS techniques [2], [12].

Machine learning based IDS which is a modern technique can detect both known and unknown threats. These systems creates models that learn patterns from network traffic data and classify them into normal or malicious categories. But it also lack interpretability, which makes it difficult to understand the decision -making process [5], [7].

Recent advancements in explainable artificial intelligence (XAI) have addressed this issue by providing insights into model predictions. SHAP (SHapely Additive exPlanations) and LIME (Local Interpretable Model -Agonistic Explanations) are techiques in XAI h elps in explaining how feature influence predictions and makes the system more reliable and transparent [17], [18].

Recent advancements in explainable artificial intelligence (XAI) have addressed this issue by providing insights into model predictions. SHAP (SHapely Additive exPlanations) and LIME (Local Interpretable Model -Agonistic Explanations) are techiques in XAI h elps in explaining how feature influence predictions and makes the system more reliable and transparent [17], [18].

This paper proposes a IDS model that combines machine learning algorithms with explainability techniques. The system is trained using the UNSW -NB15 dataset (contains modern network traffic scenarios and attack types) [1],[29].

The main contributions of this work includes Implementation of feature engineering and selection techniques, Comparison of KNN and LightGBM models, Integration of SHAP and LIME, Performance evaluation using classification metrics.

## II. LITERATURE REVIEW

A feature engineering and model optimization-based intrusion detection system using minimum Redundancy Maximum Relevance (mRMR) for feature selection and Synthetic Minority Oversampling Technique (SMOTE) for handling class imbalance is proposed in [19]. Optuna is used to optimize the CatBoost model, resulting in improved classification performance across multiple datasets. But this

## Prints number pages in pdf  and meta data

```
]: total_pages = len(reader.pages)

print(total_pages) # Gives num of pages
```

```
12
```

```
]: metadata = reader.metadata

print(metadata.title) # Gives title
print(metadata.author) # Gives author name
```

```
None
DHARSHINI G
```

## Merger the Pdf

```
: from pypdf import PdfReader, PdfWriter  # Import PDF classes

pdf1 = r"C:\Users\rajav\Downloads\VIT Downloads\Other Downloads\Assignments\Sample_pdf.pdf"  # First PDF
pdf2 = r"C:\Users\rajav\Downloads\VIT Downloads\Other Downloads\Assignments\Sample_pdf2.pdf"  # Second PDF

reader1 = PdfReader(pdf1)  # Open first PDF
reader2 = PdfReader(pdf2)  # Open second PDF

print("PDF 1 Pages :", len(reader1.pages))  # Show pages in first PDF
print("PDF 2 Pages :", len(reader2.pages))  # Show pages in second PDF

writer = PdfWriter()  # Create writer object
writer.append(pdf1)  # Add first PDF
writer.append(pdf2)  # Add second PDF

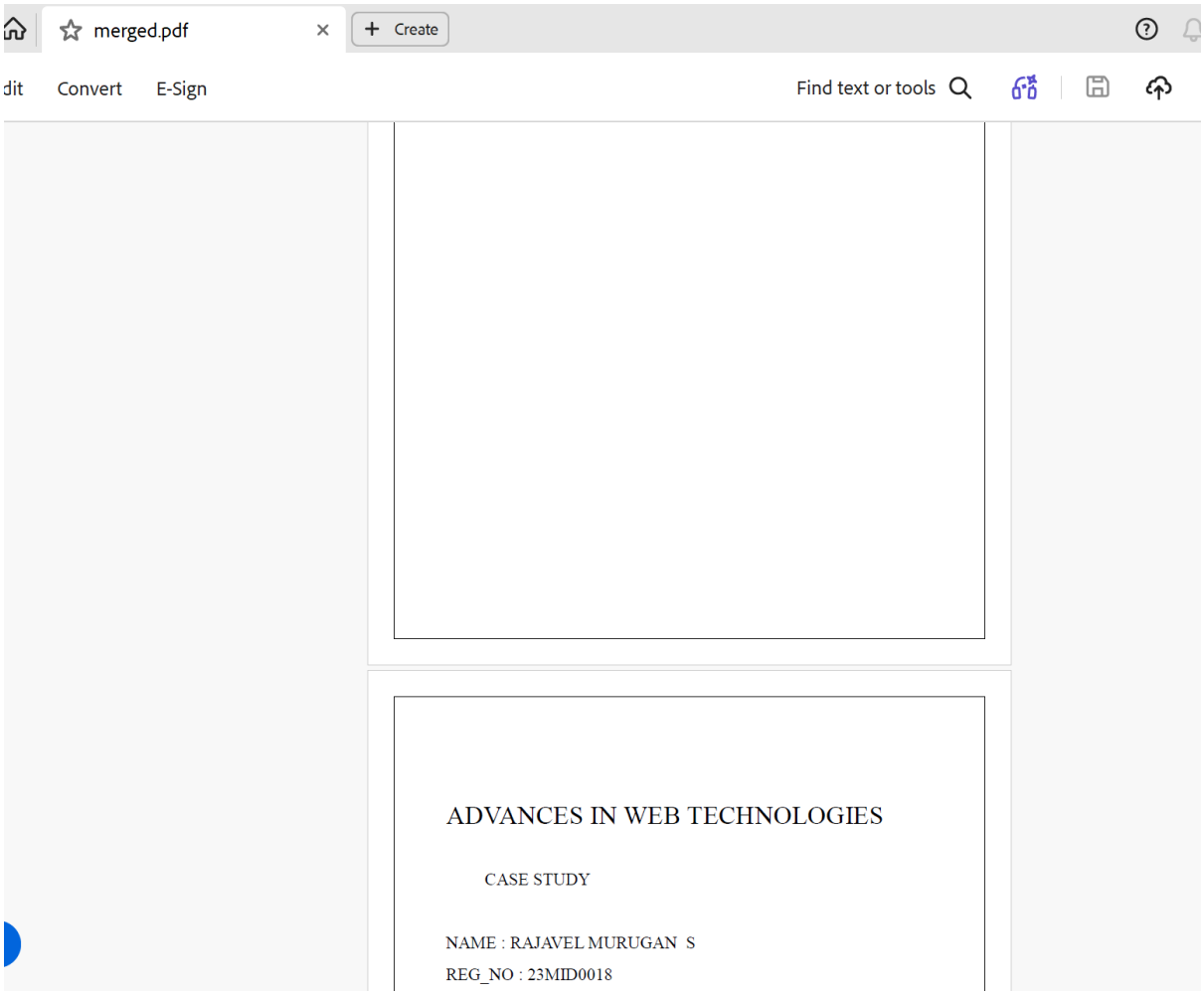
merged_file = "merged_pdf.pdf"  # Output file name

with open(merged_file, "wb") as output:  # Create output file
    writer.write(output)  # Save merged PDF

merged_reader = PdfReader(merged_file)  # Open merged PDF

print("Merged PDF Pages :", len(merged_reader.pages))  # Show total pages
print("PDFs merged successfully!")  # Display success message
```

```
PDF 1 Pages : 12
PDF 2 Pages : 10
Merged PDF Pages : 22
PDFs merged successfully!
```




## Pdf Encryption

[20]:
```python
from pypdf import PdfReader, PdfWriter  # Import PDF classes

reader = PdfReader(r"C:\Users\rajav\Downloads\VIT Downloads\Other Downloads\Assignments\Sample_pdf.pdf")  # Open PDF

writer = PdfWriter()  # Create writer object

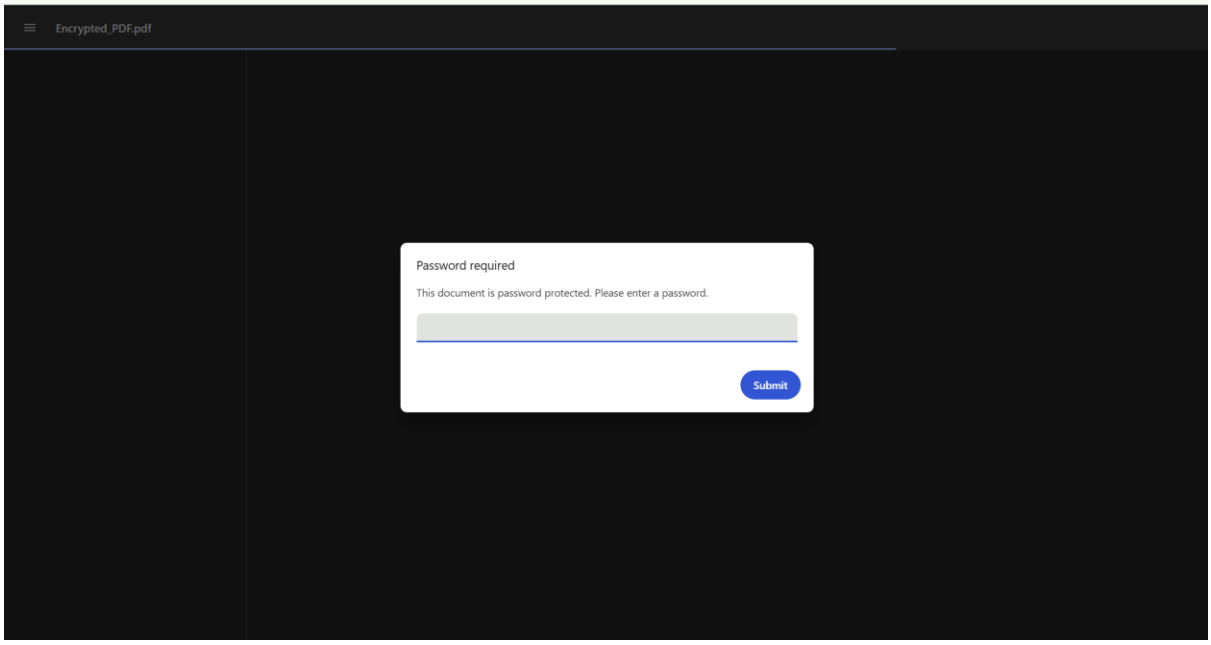
for page in reader.pages:  # Read each page
    writer.add_page(page)  # Add page to new PDF

writer.encrypt("1234")  # Set password

with open("Encrypted_PDF.pdf", "wb") as output:  # Create output file
    writer.write(output)  # Save encrypted PDF

print("PDF encrypted successfully!")  # Show success message
```

```
PDF encrypted successfully!
```


Encrypted_PDF.pdf
Password required
This document is password protected. Please enter a password.
Submit

## TASK 3: Excel File Processing Using openpyxl

```python
import openpyxl
from openpyxl.styles import Font, Alignment, Border, Side, PatternFill
from openpyxl.utils import get_column_letter


def generate_company_sales_file():
    # Create a new workbook
    wb = openpyxl.Workbook()

    # Sales Report sheet
    ws = wb.active
    ws.title = "Sales Report"
    ws.sheet_view.showGridLines = True

    # Font and colors
    FONT_FAMILY = "Arial"
    blue_header_fill = PatternFill(start_color="1B365D", end_color="1B365D", fill_type="solid")
    white_bold_font = Font(name=FONT_FAMILY, size=11, bold=True, color="FFFFFF")

    # Create borders
    thin_border_side = Side(border_style="thin", color="D3D3D3")
    double_border_side = Side(border_style="double", color="000000")

    grid_border = Border(left=thin_border_side, right=thin_border_side,
                         top=thin_border_side, bottom=thin_border_side)

    summary_border = Border(top=thin_border_side,
                            bottom=double_border_side)

    # Main title
    ws["A1"] = "Corporate Sales Ledger 2026"
    ws["A1"].font = Font(name=FONT_FAMILY, size=16, bold=True, color="1B365D")

    # Table headers
    headers = ["Item ID", "Product Name", "Unit Price", "Quantity", "Total Revenue"]

    for col_num, header_title in enumerate(headers, 1):
        cell = ws.cell(row=4, column=col_num, value=header_title)
        cell.font = white_bold_font
        cell.fill = blue_header_fill
        cell.alignment = Alignment(horizontal="center", vertical="center")
```

```python
# Sample data
mock_data = [
    ["E001", "Enterprise Server", 125000, 4],
    ["E002", "Pro Workstation", 85000, 12],
    ["E003", "Fibre Switch", 45000, 8],
    ["E004", "4K Graphics Display", 32000, 15],
    ["E005", "UPS Power Array", 18000, 20],
]

# Add data
for i, row_data in enumerate(mock_data, start=5):
    ws.cell(row=i, column=1, value=row_data[0])
    ws.cell(row=i, column=2, value=row_data[1])
    ws.cell(row=i, column=3, value=row_data[2])
    ws.cell(row=i, column=4, value=row_data[3])
    ws.cell(row=i, column=5, value=f"=C{i}*D{i}")

    # Add border and font
    for col_num in range(1, 6):
        current_cell = ws.cell(row=i, column=col_num)
        current_cell.border = grid_border
        current_cell.font = Font(name=FONT_FAMILY, size=10)

    # Format numbers
    ws.cell(row=i, column=3).number_format = '"₹"#,##0.00'
    ws.cell(row=i, column=4).number_format = '#,##0'
    ws.cell(row=i, column=5).number_format = '"₹"#,##0.00'

    # Highlight high quantity
    if row_data[3] >= 15:
        alert_fill = PatternFill(start_color="E2EFDA",
                                 end_color="E2EFDA",
                                 fill_type="solid")
        ws.cell(row=i, column=4).fill = alert_fill

# Total section
ws["A11"] = "Overall Metrics"
ws["A11"].font = Font(name=FONT_FAMILY, size=11, bold=True)

ws["D11"] = "Cumulative:"
ws["D11"].font = Font(name=FONT_FAMILY, size=11, bold=True, italic=True)
```

```python
# Total formula
ws["E11"] = "=SUM(E5:E9)"
ws["E11"].font = Font(name=FONT_FAMILY, size=11, bold=True)
ws["E11"].number_format = '"₹"#,##0.00'
ws["E11"].border = summary_border

# Adjust column width
for col in ws.columns:
    max_len = 0

    for cell in col:
        val_str = str(cell.value or '')

        if val_str.startswith('='):
            val_str = "₹1,000,000.00"

        if len(val_str) > max_len:
            max_len = len(val_str)

    col_letter = get_column_letter(col[0].column)
    ws.column_dimensions[col_letter].width = max(max_len + 5, 16)

# Dashboard sheet
ws_summary = wb.create_sheet(title="Dashboard")
ws_summary.sheet_view.showGridLines = True

ws_summary["A1"] = "Executive Performance Summary"
ws_summary["A1"].font = Font(name=FONT_FAMILY,
                             size=14,
                             bold=True,
                             color="1B365D")

# Link total from Sales Report
ws_summary["A3"] = "Total Projected Revenue Stream:"
ws_summary["B3"] = "='Sales Report'!E11"
ws_summary["B3"].font = Font(name=FONT_FAMILY,
                             size=11,
                             bold=True)

ws_summary["B3"].number_format = '"₹"#,##0.00'

# Set column width
ws_summary.column_dimensions["A"].width = 32
ws_summary.column_dimensions["B"].width = 20
```

```python
    ws_summary.column_dimensions["B"].width = 20

    # Save workbook
    file_name = "company_sales1.xlsx"
    wb.save(file_name)

    print(f"Workbook created successfully: '{file_name}'")


if __name__ == "__main__":
    generate_company_sales_file()
```

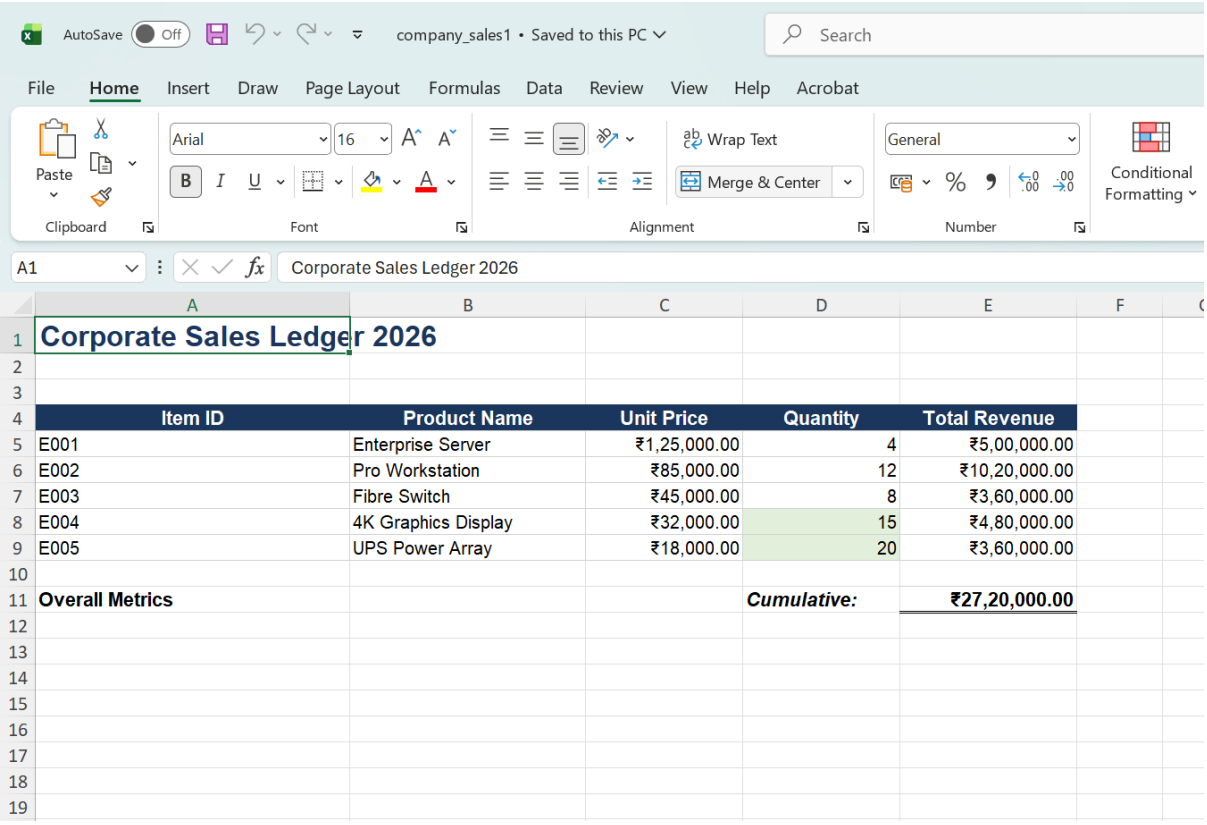
```
Workbook created successfully: 'company_sales1.xlsx'
```

Feature 1: Grid Lines & Borders (Border())

Feature 2: Number Formatting (.number_format)

Feature 3: Highlight High Quantity (PatternFill)

Feature 4: Dashboard Sheet (create_sheet())




| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | **Corporate Sales Ledger 2026** | | | | | |
| 2 | | | | | | |
| 3 | | | | | | |
| 4 | **Item ID** | **Product Name** | **Unit Price** | **Quantity** | **Total Revenue** | |
| 5 | E001 | Enterprise Server | ₹1,25,000.00 | 4 | ₹5,00,000.00 | |
| 6 | E002 | Pro Workstation | ₹85,000.00 | 12 | ₹10,20,000.00 | |
| 7 | E003 | Fibre Switch | ₹45,000.00 | 8 | ₹3,60,000.00 | |
| 8 | E004 | 4K Graphics Display | ₹32,000.00 | 15 | ₹4,80,000.00 | |
| 9 | E005 | UPS Power Array | ₹18,000.00 | 20 | ₹3,60,000.00 | |
| 10 | | | | | | |
| 11 | **Overall Metrics** | | | ***Cumulative:*** | **₹27,20,000.00** | |
| 12 | | | | | | |
| 13 | | | | | | |
| 14 | | | | | | |
| 15 | | | | | | |
| 16 | | | | | | |
| 17 | | | | | | |
| 18 | | | | | | |
| 19 | | | | | | |

## TASK 4: Audio File Processing Using Python

```
import os  # Import os module
import librosa  # Import librosa
import numpy as np  # Import numpy
import matplotlib.pyplot as plt  # Import matplotlib
from pydub import AudioSegment  # Import pydub

AUDIO_FILE = r"C:\Users\rajav\Downloads\audio1.wav"  # First audio file
AUDIO_FILE2 = r"C:\Users\rajav\Downloads\audio2.wav"  # Second audio file

# FEATURE 1 : Audio Information

def audio_information(audio_file):

    print("FEATURE 1 : AUDIO INFORMATION ")  # Display feature name

    audio, sample_rate = librosa.load(audio_file, sr=None)  # Load audio

    duration = librosa.get_duration(y=audio, sr=sample_rate)  # Get duration

    print("File Name    :", os.path.basename(audio_file))  # Show file name
    print("Duration     :", round(duration, 2), "seconds")  # Show duration
    print("Sample Rate  :", sample_rate, "Hz")  # Show sample rate
    print("Total Samples:", len(audio))  # Show total samples

    print("=" * 60)  # Print line


# FEATURE 2 : Audio Waveform (Amplitude)

def show_waveform(audio_file):

    print("FEATURE 2 : AUDIO WAVEFORM ")  # Display feature name

    audio, sample_rate = librosa.load(audio_file, sr=None)  # Load audio

    plt.figure(figsize=(10,4))  # Create graph

    plt.plot(audio)  # Plot waveform

    plt.title("Audio Waveform")  # Graph title

    plt.xlabel("Samples")  # X-axis label

    plt.ylabel("Amplitude")  # Y-axis label

    plt.grid(True)  # Show grid
```

```
    plt.show()  # Display graph

    print("=" * 60)  # Print line


# FEATURE 3 : Frequency Spectrum

def frequency_spectrum(audio_file):

    print("FEATURE 3 : FREQUENCY SPECTRUM ")  # Display feature name

    audio, sample_rate = librosa.load(audio_file, sr=None)  # Load audio

    fft = np.fft.fft(audio)  # Apply FFT

    frequency = np.fft.fftfreq(len(fft), d=1/sample_rate)  # Get frequency

    plt.figure(figsize=(10,4))  # Create graph

    plt.plot(
        frequency[:len(frequency)//2],
        np.abs(fft[:len(fft)//2])
    )  # Plot spectrum

    plt.title("Frequency Spectrum")  # Graph title

    plt.xlabel("Frequency (Hz)")  # X-axis label

    plt.ylabel("Magnitude")  # Y-axis label

    plt.grid(True)  # Show grid

    plt.show()  # Display graph

    print("=" * 60)  # Print line



# FEATURE 4 : Merge Audio Files

def merge_audio(file1, file2):

    print("FEATURE 4 : MERGE AUDIO ")  # Display feature name

    audio1 = AudioSegment.from_file(file1)  # Open first audio

    audio2 = AudioSegment.from_file(file2)  # Open second audio
```

```
    audio1 = AudioSegment.from_file(file1)  # Open first audio

    audio2 = AudioSegment.from_file(file2)  # Open second audio

    merged = audio1 + audio2  # Merge audio

    output_file = "merged_audio.wav"  # Output file

    merged.export(output_file, format="wav")  # Save merged audio

    print("Merged File :", output_file)  # Show file name

    print("=" * 60)  # Print line


# MAIN PROGRAM

if __name__ == "__main__":

    if os.path.exists(AUDIO_FILE):  # Check first audio

        audio_information(AUDIO_FILE)  # Show audio information

        show_waveform(AUDIO_FILE)  # Display waveform

        frequency_spectrum(AUDIO_FILE)  # Display frequency spectrum

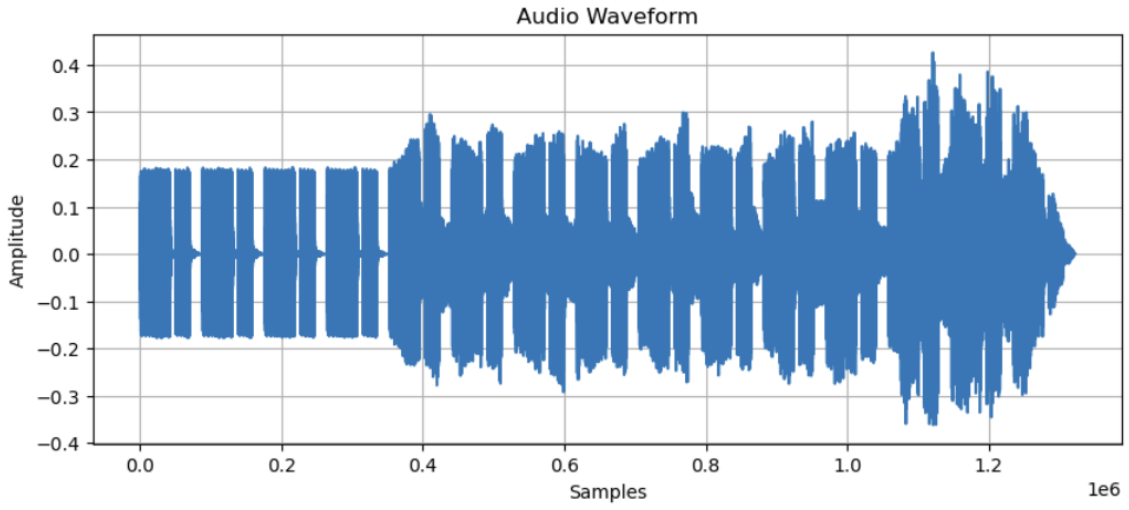
        if os.path.exists(AUDIO_FILE2):  # Check second audio

            merge_audio(AUDIO_FILE, AUDIO_FILE2)  # Merge audio files

        else:
            print("Second audio file not found.")  # Show error

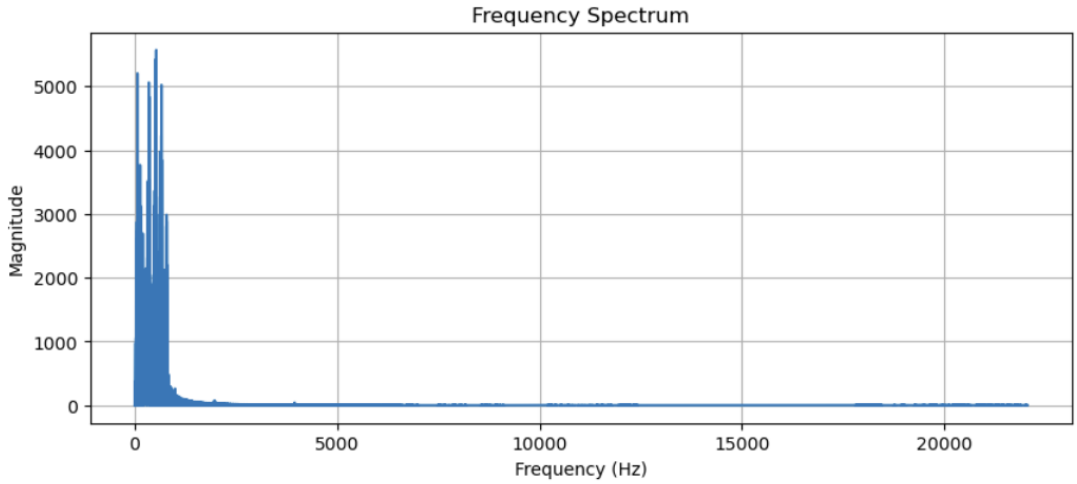
    else:
        print("Audio file not found.")  # Show error
```

```
FEATURE 1 : AUDIO INFORMATION
File Name    : audio1.wav
Duration     : 29.98 seconds
Sample Rate  : 44100 Hz
Total Samples: 1322253
============================================================
FEATURE 2 : AUDIO WAVEFORM
```

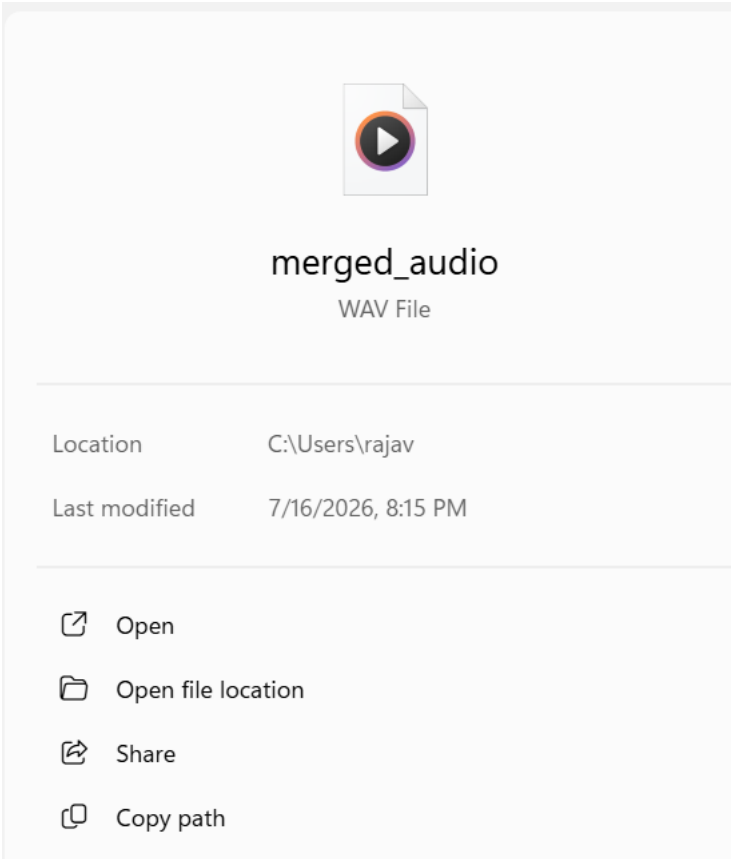


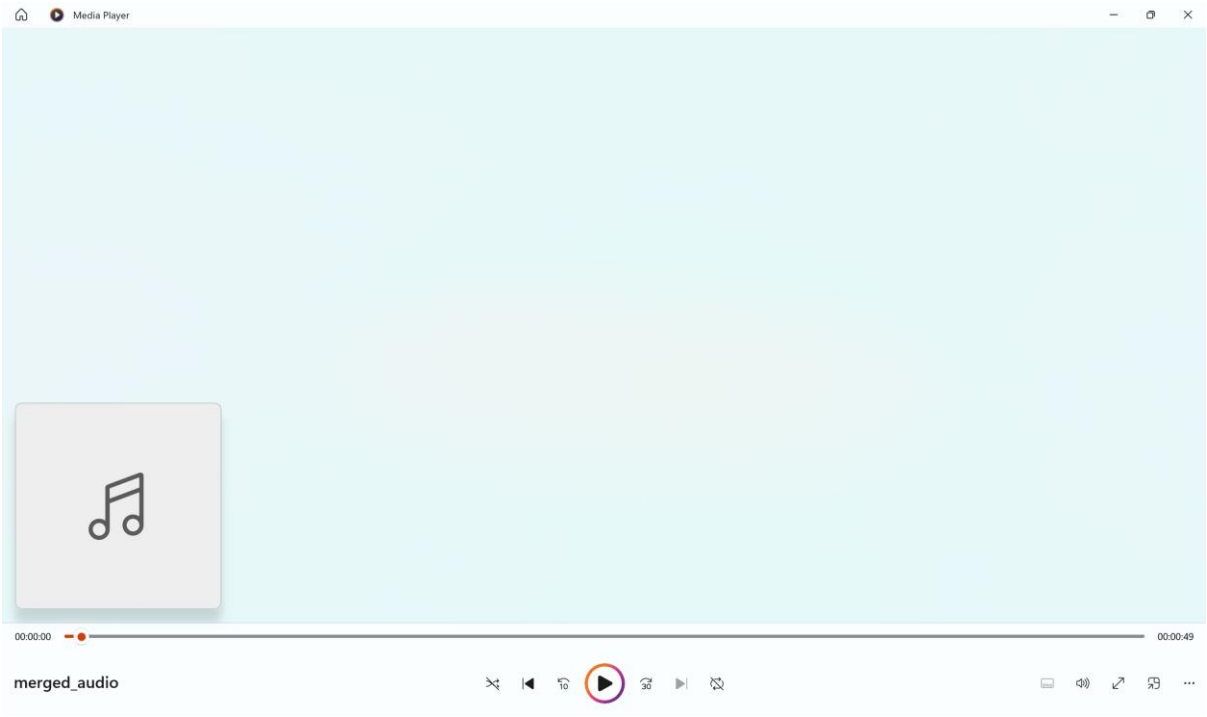
Samples

==========
FEATURE 3 : FREQUENCY SPECTRUM




==========
FEATURE 4 : MERGE AUDIO
Merged File : merged_audio.wav
==========




Media Player
00:00:00
00:00:49
merged_audio

## TASK 5: Web Scraping Using BeautifulSoup and Requests

```
]: import requests  # Import requests module
   from bs4 import BeautifulSoup  # Import BeautifulSoup

   URL = "https://books.toscrape.com/"  # Website URL

   # FEATURE 1 : Fetch Web Page

   def fetch_webpage(url):

       print("FEATURE 1 : FETCH WEB PAGE")  # Display feature name

       response = requests.get(url)  # Send request

       print("Status Code :", response.status_code)  # Show status code

       if response.status_code == 200:
           print("Web page loaded successfully.")  # Success message
       else:
           print("Failed to load web page.")  # Error message

       print("=" * 60)  # Print line

       return response  # Return response

   # FEATURE 2 : Extract Web Page Content

   def extract_content(response):

       print("FEATURE 2 : EXTRACT WEB PAGE CONTENT")  # Display feature name

       soup = BeautifulSoup(response.text, "html.parser")  # Parse HTML

       print("\nTitle:")
       if soup.title:
           print(soup.title.string)  # Show page title

       print("\nHeadings:")
       for heading in soup.find_all(["h1", "h2", "h3"]):
           print("-", heading.get_text(strip=True))  # Show headings

       print("\nParagraphs:")
       for para in soup.find_all("p"):
           print(para.get_text(strip=True))  # Show paragraphs

       print("=" * 60)  # Print line
```

```python
# FEATURE 3 : Extract Hyperlinks

def extract_links(response):

    print("FEATURE 3 : EXTRACT HYPERLINKS ")  # Display feature name

    soup = BeautifulSoup(response.text, "html.parser")  # Parse HTML

    links = soup.find_all("a")  # Find all links

    print("Total Links :", len(links))  # Show number of links

    for link in links:
        text = link.get_text(strip=True)  # Get link text
        href = link.get("href")  # Get URL

        print("Text :", text)
        print("URL  :", href)
        print()

    print("=" * 60)  # Print line

# FEATURE 4 : Website Permissions

def website_permissions(response, url):

    print(" FEATURE 4 : WEBSITE PERMISSIONS ")  # Display feature name

    print("Status Code :", response.status_code)  # Show status code

    print("Content Type :", response.headers.get("Content-Type"))  # Show content type

    print("Server :", response.headers.get("Server"))  # Show server

    print("Encoding :", response.encoding)  # Show encoding

    robots_url = url.rstrip("/") + "/robots.txt"  # robots.txt URL

    robots = requests.get(robots_url)  # Open robots.txt

    print("\nrobots.txt URL :", robots_url)  # Show robots URL

    if robots.status_code == 200:
        print("robots.txt : Available")  # robots.txt found
        print("\nFirst 500 Characters:\n")
        print(robots.text[:500])  # Show first 500 characters
    else:
        print("robots.txt : Not Available")  # robots.txt not found
```

```
    print("Server :", response.headers.get("Server"))  # Show server

    print("Encoding :", response.encoding)  # Show encoding

    robots_url = url.rstrip("/") + "/robots.txt"  # robots.txt URL

    robots = requests.get(robots_url)  # Open robots.txt

    print("\nrobots.txt URL :", robots_url)  # Show robots URL

    if robots.status_code == 200:
        print("robots.txt : Available")  # robots.txt found
        print("\nFirst 500 Characters:\n")
        print(robots.text[:500])  # Show first 500 characters
    else:
        print("robots.txt : Not Available")  # robots.txt not found

    print("=" * 60)  # Print line


# MAIN PROGRAM

if __name__ == "__main__":

    response = fetch_webpage(URL)  # Fetch web page

    if response.status_code == 200:  # Check response

        extract_content(response)  # Extract page content

        extract_links(response)  # Extract hyperlinks

        website_permissions(response, URL)  # Show website permissions

    else:

        print("Cannot access the website.")  # Display error
```

FEATURE 1 : FETCH WEB PAGE
Status Code : 200
Web page loaded successfully.
============================================================
FEATURE 2 : EXTRACT WEB PAGE CONTENT

Title:

    All products | Books to Scrape - Sandbox


Headings:
- All products
- A Light in the ...
- Tipping the Velvet
- Soumission
- Sharp Objects
- Sapiens: A Brief History ...
- The Requiem Red
- The Dirty Little Secrets ...
- The Coming Woman: A ...
- The Boys in the ...
- The Black Maria
- Starving Hearts (Triangular Trade ...
- Shakespeare's Sonnets
- Set Me Free
- Scott Pilgrim's Precious Little ...
- Rip it Up and ...
- Our Band Could Be ...
- Olio
- Mesaerion: The Best Science ...
- Libertarianism for Beginners
- It's Only the Himalayas

Paragraphs:

Â£51.77
In stock

Â£53.74
In stock

Â£50.10
In stock

Â£47.82
In stock

Â£54.23
In stock

```
============================================================
FEATURE 3 : EXTRACT HYPERLINKS
Total Links : 94
Text : Books to Scrape
URL  : index.html

Text : Home
URL  : index.html

Text : Books
URL  : catalogue/category/books_1/index.html

Text : Travel
URL  : catalogue/category/books/travel_2/index.html

Text : Mystery
URL  : catalogue/category/books/mystery_3/index.html

Text : Historical Fiction
URL  : catalogue/category/books/historical-fiction_4/index.html

Text : Sequential Art
URL  : catalogue/category/books/sequential-art_5/index.html

Text : Classics
URL  : catalogue/category/books/classics_6/index.html

Text : Philosophy
URL  : catalogue/category/books/philosophy_7/index.html

Text : Romance
URL  : catalogue/category/books/romance_8/index.html

Text : Womens Fiction
URL  : catalogue/category/books/womens-fiction_9/index.html

Text : Fiction
URL  : catalogue/category/books/fiction_10/index.html

Text : Childrens
URL  : catalogue/category/books/childrens_11/index.html

Text : Religion
URL  : catalogue/category/books/religion_12/index.html

Text : Nonfiction
URL  : catalogue/category/books/nonfiction_13/index.html


    Text :
    URL  : catalogue/libertarianism-for-beginners_982/index.html

    Text : Libertarianism for Beginners
    URL  : catalogue/libertarianism-for-beginners_982/index.html

    Text :
    URL  : catalogue/its-only-the-himalayas_981/index.html

    Text : It's Only the Himalayas
    URL  : catalogue/its-only-the-himalayas_981/index.html

    Text : next
    URL  : catalogue/page-2.html

    ============================================================
     FEATURE 4 : WEBSITE PERMISSIONS
    Status Code : 200
    Content Type : text/html
    Server : None
    Encoding : ISO-8859-1

    robots.txt URL : https://books.toscrape.com/robots.txt
    robots.txt : Not Available
    ============================================================
```